

Overview of the SliceFS Repository

<https://gemini.google.com/app/9377c123219c7103>

User prompt: can you check this repository and tell me about it?

Response: Based on the repository page you're viewing, here is an overview of [SliceFS](#):

- **What it is:** It's described as a **content-addressed FUSE filesystem**. FUSE (Filesystem in Userspace) allows users to create their own file systems without editing kernel code, and "content-addressed" typically means that files are stored and retrieved based on a hash of their contents rather than a traditional file path.
- **Language:** It appears to be written in **Rust**, as indicated by the presence of the `.cargo` and `crates` directories.
- **License:** It is distributed under the **AGPL-3.0 license**.
- **Development Status:** The project has **244 commits**, with the most recent activity visible from about 5 to 6 months ago. Some of the recent commit messages indicate that it includes benchmark infrastructure, GitHub Actions CI for Linux, and support for macOS mounting via FUSE-T.
- **Popularity:** It is currently a very niche or personal project, showing 0 stars and 0 forks.

Would you like me to look deeper into any specific folders like the `crates` or `.planning` directories to see how it's structured?

User prompt: I wanna you to consider how the tech within can be used as a base for git implementation, instead of git files/dirs.

Response: This is a fascinating thought experiment because **Git actually is a content-addressed tracker under the hood**, but it has a major bottleneck: it has to constantly translate its internal content-addressed database (`.git/objects`) into a standard POSIX filesystem (your working directory) so that you and your tools can actually interact with the files.

Using a content-addressed FUSE (Filesystem in Userspace) implementation like [SliceFS](#) as the base for a version control system would fundamentally change how working directories operate.

Here is how replacing Git's standard file/directory materialization with a SliceFS-style virtual filesystem could revolutionize a Git-like tool:

1. Instant, Zero-Copy Checkouts (O(1) Branch Switching)

Currently, when you run `git checkout [branch]`, Git has to physically delete files, create new ones, and rewrite contents in your working directory to match the target commit. In a massive repository, this takes time and disk I/O.

- **With a FUSE CAS (Content-Addressed Storage):** Your working directory is just a virtual mount. Checking out a branch would just mean telling the FUSE driver, "*Hey, the new root*

hash is now xyz123." The checkout would be instantaneous, regardless of whether the repository has 10 files or 10 million files.

2. Native, Transparent "Lazy Fetching" for Monorepos

Massive codebases (like those at Google, Meta, or Microsoft) struggle with Git because cloning and maintaining the working tree requires downloading terabytes of data. Microsoft built *VFS for Git* (Virtual File System for Git) to solve exactly this problem using similar concepts.

- **With a FUSE CAS:** You could clone a 500GB repository instantly. The FUSE mount would make it *look* like all the files are there. The actual file data would only be downloaded from the remote server over the network at the exact moment you (or your IDE) try to open a specific file.

3. Sub-File (Block-Level) Deduplication

Git tracks content at the *file* level. If you have a 1GB database dump and you change one line, Git creates a completely new 1GB blob in `.git/objects` (though it uses packfiles to compress these deltas later, the initial creation is inefficient).

- **With a FUSE CAS:** Filesystems usually break data into chunks or blocks (e.g., 4KB, 1MB slices). If SliceFS addresses content at the block/slice level, modifying that 1GB file would only require storing the single new block that changed, plus a new file manifest. This makes handling massive binaries, databases, and media files trivial—solving Git's infamous weakness with large files without needing a clunky bolt-on like Git LFS.

4. Ephemeral and Cheap Worktrees

In Git, if you want to work on three branches at once without stashing, you use `git worktree`, which physically duplicates the checked-out files on your hard drive, eating up space.

- **With a FUSE CAS:** You could spin up 50 different working directories mounted to 50 different commits. Because they are all virtual and backed by the same content-addressed block store, they would consume zero extra disk space for the file contents.

The Catch: The Performance Trade-off

The main reason standard Git doesn't do this natively is **FUSE overhead**. Every time a compiler, IDE, or script wants to read a file, the request has to travel from User Space -> Kernel Space -> User Space (FUSE driver) -> Kernel Space -> Disk. While instant checkouts and lazy loading are amazing, running a massive `npm install` or compiling a massive C++ project inside a FUSE mount can sometimes be noticeably slower than on a native filesystem (like ext4 or APFS) due to the system call overhead.

However, seeing that SliceFS is being built in Rust and includes benchmark infrastructure, it seems the author is very likely exploring these exact performance boundaries!

User prompt: It can be ported to OS core, if necessary.

Response: If you port a content-addressed storage (CAS) filesystem like **SliceFS** directly into the **OS kernel**—as a native kernel-space filesystem (similar to Btrfs, ZFS, or APFS)—you eliminate the single biggest drawback of FUSE: performance overhead.

Eliminating context-switching between user space and kernel space turns this concept from an experimental approach into a **next-generation native version-controlled filesystem**.

Here is how moving this technology into the OS core transforms a Git-like system:

1. Near-Native I/O Performance

- **No Context-Switch Penalty:** In FUSE, every read/write system call travels `User Space → Kernel → FUSE Daemon → Kernel → Disk`. In the kernel, the VFS (Virtual File System) layer talks directly to the block layer or page cache.
- **Lightning-Fast Builds:** Massive I/O operations—like running `npm install`, `cargo build`, or compiling C++ projects—happen at raw disk speed while maintaining full versioning.

2. Global Page Cache Deduplication

When the OS kernel manages block-level content addressing:

- **RAM Optimization:** If five different branches or working trees share the exact same 4KB data blocks, the kernel's **page cache** only needs to store that block in system RAM *once*.
- **Shared Memory:** Running multiple test environments or build workers simultaneously consumes fractionally less memory because identical binaries, dependencies, and source files share the same cached memory pages.

3. Syscall-Level Version Control (`ioctl` / Native API)

Instead of running a user-space tool that scans directory trees and writes index files, Git commands could interact directly with the kernel via `ioctl` or dedicated system calls:

- **checkout(hash):** The kernel simply updates the mount's root inode hash in memory. Switching branches on a multi-gigabyte repo takes **microseconds**.
- **commit():** The kernel freezes the current transactional state of the block tree, hashes the modified blocks, and returns a new root tree hash—completely bypassing the need for a user-space staging index (`.git/index`).

4. Continuous, Invisible Snapshotting

Kernel-level content addressing naturally bridges the gap between **file history** and **system snapshots**:

- **Crash Recovery & Time Travel:** Every file write can be copy-on-write (CoW). If a build script corrupts a file or a test suite modifies data, you can roll back the working tree to the exact millisecond before the command was executed.

- **Zero-Cost Worktrees:** Spawning a new working directory for a feature branch or CI worker is just a lightweight pointer alias in the kernel, taking **0 bytes of extra disk space** until new writes occur.

Real-World Engineering Trade-offs

Aspect	User-Space (FUSE)	OS Core (Kernel Driver)
I/O Speed	Moderate (IPC overhead)	Maximum (Direct VFS access)
Portability	High (Works across Linux, macOS, BSD)	Low (Requires dedicated drivers per OS)
Safety / Stability	High (Crash only kills daemon)	Risk (Bug can panic/crash the whole OS)
Deployment	Easy (Install user binary)	Hard (Requires kernel module / upstream OS support)

Existing Parallels

This vision is similar to what **Apple's APFS** (with local snapshots/clones), **Btrfs**, and **ZFS** do at the filesystem level, combined with **Meta's Sapling** / **Microsoft's VFS for Git** at the VCS layer.

Building a CAS-first filesystem in **Rust for the Linux kernel** (a direction the Linux kernel is actively moving toward) could be a path to making deep, kernel-native version control a reality.

Would you like to explore how data structures (like Merkle trees or DAGs) would handle diffing and merging natively at the block level?

User prompt: Can the whole SliceFS or some parts of it be moved into NVME?

Response: Yes, absolutely. Pushing a content-addressed storage (CAS) system like SliceFS directly into the NVMe hardware is not only possible, but it aligns perfectly with an emerging frontier in enterprise storage known as **Computational Storage** and **NVMe-KV**.

By moving parts of—or the entirety of—the filesystem logic down to the NVMe controller, you bypass the host CPU, OS kernel, and system RAM almost entirely.

Here is how the architecture of a CAS filesystem like SliceFS could be mapped directly into NVMe hardware:

1. NVMe Key-Value Command Set (NVMe-KV)

Standard SSDs use **Logical Block Addressing (LBA)**—the OS says, "put this data in block 500." A CAS filesystem has to maintain a massive mapping table converting file hashes (e.g., a1b2c3...) to LBAs.

- **How it moves to NVMe:** The SNIA standard for **NVMe-KV** changes how drives accept data. Instead of LBAs, the drive natively accepts variably sized objects mapped directly to a cryptographic hash key.
- **The Result:** The entire deduplication lookup table and block allocation logic is offloaded to the drive's internal Flash Translation Layer (FTL). You hand the NVMe drive a 32-byte hash and a chunk of code, and the drive handles the physical placement and deduplication natively.

2. NVMe Computational Storage Devices (CSDs)

Modern "SmartSSDs" (like those from Samsung or NGD Systems) include built-in ARM cores or FPGAs directly on the NVMe controller to process data before it ever crosses the PCIe bus to your motherboard.

- **Hashing Offload:** The most CPU-intensive part of SliceFS or Git is calculating the cryptographic hashes (SHA-1, SHA-256, or BLAKE3) of every file and slice. An NVMe CSD with an FPGA can calculate these hashes at the speed of the raw flash media.
- **Compression Offload:** When you commit data, the NVMe controller can compress it on the fly, hash it, and store it without your CPU doing a single clock cycle of work.

3. Controller Memory Buffer (CMB) for Instant Merkle Trees

To navigate a Git repository or a SliceFS tree, the system has to traverse a Merkle tree (a tree of hashes pointing to other hashes).

- **How it moves to NVMe:** NVMe supports **CMB** and **PMR** (Persistent Memory Region), which allows the drive to expose a chunk of its internal high-speed RAM/NVRAM directly to the host OS.
- **The Result:** The entire metadata tree (directories, file manifests, and commit history) could live inside the NVMe drive's CMB. Checking out a massive repository would mean updating pointers inside the drive's own memory, making branch switching practically instantaneous with zero latency penalty from the PCIe bus.

What this means for a Git-like System

If you combine these three technologies—using NVMe-KV to store blobs and NVMe CSDs to compute hashes—you get a version control system that operates at the hardware level:

- **Zero-Copy Commits:** You tell the drive, "Snapshot this directory." The drive computes the hashes internally, deduplicates against what it already holds in its flash memory, and returns the root commit hash. The host CPU is virtually idle.
- **Bus-Free Garbage Collection:** Finding "orphaned" git objects (data no longer linked to any branch) usually requires standard Git to pull gigabytes of data into host RAM to compare. An NVMe CSD can run garbage collection natively in the background without moving any data across the motherboard.

The Barrier to Entry

The only reason we don't build software this way today is **hardware lock-in**. Taking SliceFS and porting it to NVMe-KV or SmartSSD APIs means your filesystem will *only* run on specialized enterprise data-center hardware, abandoning compatibility with standard laptops and consumer SSDs. However, for extreme-scale monorepos or massive data-versioning pipelines, this hardware-accelerated CAS approach is exactly where the industry is heading.

User prompt: 1. the backend can be abstracted out, so it either works as a module in the host OS kernel, or within NVME, depending on the configuration and/or available hardware.

Response: This is exactly the right architectural approach. In systems engineering, this is known as an **"hourglass architecture"**—a broad set of user-facing commands (like `checkout`, `commit`, `status`) funnel down into a narrow, standardized API, which then fans out to multiple interchangeable hardware/software backends.

By abstracting the backend, you create a system that elegantly gracefully degrades: it runs at mind-bending hardware speeds on enterprise NVMe Computational Storage Devices (CSDs), but still functions perfectly well as a standard kernel module on a consumer laptop.

Here is how you would design and implement this abstraction layer using modern storage standards:

1. The Narrow Waist: The Unified Storage API

To make the backend interchangeable, the core VCS logic cannot use standard POSIX file commands like `open()` or `read()`. Instead, it must communicate through a unified message-passing API designed for content-addressed storage (CAS) and computational offload.

Libraries like **xNVMe** or the **SNIA Computational Storage API** already exist to provide this exact abstraction. Your system would interface via a Rust trait (e.g., `StorageEngine`) that defines operations like:

- `put_object(data) -> hash` (Write data, let the backend handle hashing and deduplication)
- `get_object(hash) -> data` (Retrieve data by hash)
- `compute_diff(hash_A, hash_B) -> patch` (Ask the backend to compare two trees)

2. Execution Path A: Host OS Kernel Module (The Software Fallback)

If the system is running on a standard consumer laptop or virtual machine, the abstraction layer routes the commands to a host-side kernel module.

- **Implementation:** Thanks to the **Rust for Linux** project (which officially moved out of experimental status and integrated NVMe driver efforts), this module can be written entirely in safe Rust.
- **How it works:** When the API calls `put_object()`, the host CPU calculates the cryptographic hash (e.g., BLAKE3). The kernel module then checks its mapping table in

system RAM, deduplicates the block, and writes it to a standard NVMe SSD using highly asynchronous `io_uring` commands.

- **Advantage:** Unlocks massive performance gains over user-space FUSE by utilizing the kernel's native page cache and eliminating context switches, while requiring no specialized hardware.

3. Execution Path B: NVMe CSD (The Hardware Offload)

If the abstraction layer detects an enterprise **NVMe Computational Storage Device (CSD)** or an NVMe Key-Value (NVMe-KV) drive, it bypasses the host kernel's compute entirely.

- **Implementation:** The system uses frameworks like **SPDK** (Storage Performance Development Kit) to bypass the OS kernel and talk directly to the PCIe bus.
- **How it works:** When the API calls `put_object()`, the host sends the *raw, unhashed data* directly to the drive. The drive's internal ARM core or FPGA calculates the hash, handles the deduplication, and stores the object.
- **Advantage:** The host CPU does zero work. You can commit gigabytes of data, and the host processor won't even spin up its fans.

4. Transparent Portability via eBPF or WebAssembly

One of the most exciting developments in the SNIA Computational Storage standard is how compute instructions are shipped to the drives. Instead of hardcoding Git logic into the NVMe firmware, the abstraction layer can compile the diffing, merging, and tree-traversal algorithms into **eBPF** (Extended Berkeley Packet Filter) or **WebAssembly (Wasm)**.

When you initialize the repository, the abstraction layer simply asks: *"Are you a Smart NVMe drive?"*

- If **Yes**, it pushes the Wasm/eBPF binary down to the drive's firmware to execute locally.
- If **No**, it executes the exact same Wasm/eBPF binary in the host OS kernel.

This guarantees that the version control logic is perfectly consistent across both environments, with zero rewrite required when upgrading hardware.

Sources

1. [Rust in the Linux Kernel: An Embedded Engineer's View](#)
2. [SPDK — xNVMe v0.7.5 documentation](#)

User prompt: find me NVME drives that support these

Response: Here are specific, real-world enterprise NVMe drives and prototypes that support the advanced storage architectures we discussed, broken down by their specific capabilities:

1. NVMe Computational Storage Drives (CSDs)

These drives contain onboard processors (ARM cores or FPGAs) to manipulate, compress, or hash data before it ever touches the host CPU.

- **ScaleFlux CSD 3000 Series**

- **How it works:** This drive features a custom SoC ARM chip (the SFX 3000 Storage Processor) built directly into the controller. It transparently handles data compression and decompression at line-rate speed.
- **Git/CAS Application:** In a version control system, this drive could automatically compress Git objects as they are written, requiring absolutely zero CPU cycles from the host machine.

- **Samsung / Xilinx (AMD) SmartSSD**

- **How it works:** This is the most famous programmable CSD. It combines standard Samsung V-NAND with a massive **Xilinx Kintex Ultrascale+ FPGA** directly on the drive.
- **Git/CAS Application:** Because it has a programmable FPGA, you could literally flash a hardware-accelerated BLAKE3 or SHA-256 hashing algorithm directly onto the drive. You hand the drive raw data, and the FPGA computes the commit hash natively.

- **NGD Systems Newport / Catalina Series**

- **How it works:** NGD was a pioneer in CSDs, putting quad-core ARM Cortex-A53 processors directly onto the NVMe controller. Instead of a fixed-function FPGA, these drives run a localized version of Linux internally. You can push containerized applications (like an eBPF diffing script) directly to the drive to execute on the flash media. *(Note: NGD as a standalone company has faced recent corporate changes, but their hardware defined the SNIA standard).*

2. NVMe Key-Value (KV) Drives

These drives abandon standard Logical Block Addressing (LBA) and natively accept data mapped to a cryptographic hash or key, handling the internal mapping table themselves.

- **Samsung KV-SSD (e.g., PM983 KV / PM9A3 KV variants)**

- **How it works:** Samsung built the first major enterprise SSDs that natively speak the SNIA Key-Value API. Instead of the OS asking to write to block 0x500, the OS tells the drive to write an object associated with a key (like a 32-byte Git hash).
- **Git/CAS Application:** The drive's Flash Translation Layer (FTL) handles all deduplication, garbage collection, and object mapping natively. The host OS doesn't need to maintain a `.git/objects` folder structure at all.

3. NVMe with Controller Memory Buffer (CMB)

CMB is a standard feature in the NVMe specification (since NVMe 1.2), but it is generally only enabled on high-end enterprise hardware. It allows the host PC to use a portion of the SSD's internal ultra-fast DRAM for command queues or data buffering.

- **Intel / Solidigm D7-Series (e.g., D7-P5510, P5520)**
- **Samsung PM1733 / PM9A3 Datacenter SSDs**
 - **How they work:** These standard enterprise drives support mapping their internal PCIe RAM directly into the host OS's memory space.
 - **Git/CAS Application:** You could store the "Merkle tree" (the directory structure and commit graphs of the Git repository) directly in the drive's CMB. When traversing the commit history or running a `git log`, the system reads the tree layout from the drive's internal RAM at nanosecond latencies without waking up the main storage flash at all.

How to actually get them: While standard drives with CMB (like the Solidigm D7) can be purchased easily from enterprise vendors, true Computational Storage Drives (like the ScaleFlux CSD 3000 or the Samsung SmartSSD) and Key-Value drives are strictly B2B enterprise products. They are typically sold directly to hyperscalers (AWS, Meta, Azure) or specialized data center integrators.

Sources

1. [ScaleFlux CSD 3000 SSD Review - StorageReview.com](#)
 2. [Get better utilization, efficiency and TCO with ScaleFlux CSD 3000](#)
 3. [3.84 TB - U.2 PCIe 3.0 x4 \(NVMe\) - Xilinx Samsung SmartSSD](#)
-